

DBMS Lab Assignment – 7

SQL Procedures

PL/SQL subprograms are named PL/SQL blocks that can be invoked with a set of parameters. PL/SQL provides two kinds of subprograms:

- **Functions:** these subprograms return a single value, mainly used to compute and return a value.
- **Procedures:** these subprograms do not return a value directly, mainly used to perform an action.

This chapter is going to cover important aspects of a **PL/SQL procedure** and we will cover **PL/SQL function** in next chapter.

Parts of a PL/SQL Subprogram

Each PL/SQL subprogram has a name, and may have a parameter list. Like anonymous PL/SQL blocks and, the named blocks a subprograms will also have following three parts:

Sr.No	Parts & Description
1	Declarative Part:- It is an optional part. However, the declarative part for a subprogram does not start with the DECLARE keyword. It contains declarations of types, cursors, constants, variables, exceptions, and nested subprograms. These items are local to the subprogram and cease to exist when the subprogram completes execution.
2	Executable Part: - This is a mandatory part and contains statements that perform the designated action.
3	Exception-handling :- This is again an optional part. It contains the code that handles run-time errors.

Creating a Procedure

A procedure is created with the CREATE OR REPLACE PROCEDURE statement. The simplified syntax for the CREATE OR REPLACE PROCEDURE statement is as follows:

```

CREATE [OR REPLACE] PROCEDURE procedure_name
[(parameter_name [IN | OUT | IN OUT] type [, ...])]
{IS | AS}
BEGIN
    < procedure_body >

```

Where,

- *procedure-name* specifies the name of the procedure.
- [OR REPLACE] option allows modifying an existing procedure.
- The optional parameter list contains name, mode and types of the parameters. IN represents that value will be passed from outside and OUT represents that this parameter will be used to return a value outside of the procedure.
- *procedure-body* contains the executable part.
- The AS keyword is used instead of the IS keyword for creating a standalone procedure.

S. N.	Parameter Mode & Description
1	IN :- An IN parameter lets you pass a value to the subprogram. It is a read-only parameter. Inside the subprogram, an IN parameter acts like a constant. It cannot be assigned a value. You can pass a constant, literal, initialized variable, or expression as an IN parameter. You can also initialize it to a default value; however, in that case, it is omitted from the subprogram call. It is the default mode of parameter passing. Parameters are passed by reference.
2	OUT:- An OUT parameter returns a value to the calling program. Inside the subprogram, an OUT parameter acts like a variable. You can change its value and reference the value after assigning it. The actual parameter must be variable and it is passed by value.

- 3 **IN OUT:-** An IN OUT parameter passes an initial value to a subprogram and returns an updated value to the caller. It can be assigned a value and its value can be read.
- The actual parameter corresponding to an IN OUT formal parameter must be a variable, not a constant or an expression. Formal parameter must be assigned a value.
- Actual parameter is passed by value.**

Assignment 7 Lab Questions

Implementation Questions:

1. Write a procedure to find details of books of a given library.

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE get_library_books(IN p_lib_id INT)
-> BEGIN
-> SELECT
-> Bid,
-> Bname,
-> Price,
-> Author,
-> Publisher,
-> Copies
-> FROM books
-> WHERE Lid = p_lib_id;
-> END //
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> CALL get_library_books(1);
+-----+-----+-----+-----+-----+
| Bid | Bname                                     | Price | Author | Publisher          | Copies |
+-----+-----+-----+-----+-----+
| 101 | Fundamentals of Database Systems         | 650.00 | Navathe | Pearson Education  | 15     |
| 102 | Database System Concepts                | 720.00 | Korth   | McGraw Hill        | 20     |
| 103 | Operating System Concepts               | 890.00 | Galvin  | Wiley India         | 12     |
| 104 | Advanced Database Management            | 450.00 | Navathe | Pearson Education  | 8      |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
```


3. Write a procedure to increase price of books by 10% which are sold by a particular seller to a particular library.

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE increase_book_price_seller(
->     IN p_seller_id INT,
->     IN p_lib_id INT
-> )
-> BEGIN
->     UPDATE books b
->     JOIN purchase p ON b.Bid = p.Bid
->     SET b.Price = b.Price * 1.10
->     WHERE p.Sid = p_seller_id
->         AND p.Lid = p_lib_id;
->
->     SELECT ROW_COUNT() AS updated_books_count;
-> END //
```

Query OK, 0 rows affected (0.00 sec)

```
mysql>
mysql> DELIMITER ;
mysql>
mysql> CALL increase_book_price_seller(1, 1);
```

```
+-----+
| updated_books_count |
+-----+
|                    1 |
+-----+
1 row in set (0.01 sec)
```

Query OK, 0 rows affected (0.01 sec)

```
mysql> SELECT Bid, Bname, Price FROM books WHERE Bid = 101;
+-----+-----+-----+
| Bid | Bname                                | Price |
+-----+-----+-----+
| 101 | Fundamentals of Database Systems | 715.00 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

4. Write a procedure to count total number of publishers who are providing books to a particular library.

```
mysql>
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE count_publishers_for_lib(
->     IN p_lib_id INT,
->     OUT p_pub_count INT
-> )
-> BEGIN
->     SELECT COUNT(DISTINCT Pid)
->     INTO p_pub_count
->     FROM purchase
->     WHERE Lid = p_lib_id;
-> END //
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> CALL count_publishers_for_lib(1, @pub_count);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT @pub_count AS total_publishers;
+-----+
| total_publishers |
+-----+
|                1 |
+-----+
1 row in set (0.00 sec)
```

5. Write a procedure to find the costliest and the cheapest books that a particular library has.

```
mysql> DELIMITER //
```

```
mysql>
```

```
mysql> CREATE PROCEDURE get_extreme_priced_books(IN p_lib_id INT)
```

```
  -> BEGIN
```

```
  ->     -- Costliest book
```

```
  ->     (SELECT 'Costliest' AS Category, Bid, Bname, Price
```

```
  ->       FROM books
```

```
  ->       WHERE Lid = p_lib_id
```

```
  ->       ORDER BY Price DESC
```

```
  ->       LIMIT 1)
```

```
  ->     UNION ALL
```

```
  ->     -- Cheapest book
```

```
  ->     (SELECT 'Cheapest' AS Category, Bid, Bname, Price
```

```
  ->       FROM books
```

```
  ->       WHERE Lid = p_lib_id
```

```
  ->       ORDER BY Price ASC
```

```
  ->       LIMIT 1);
```

```
  -> END //
```

```
Query OK, 0 rows affected (0.00 sec)
```



```
mysql>
```

```
mysql> DELIMITER ;
```

```
mysql>
```

```
mysql> CALL get_extreme_priced_books(1);
```

Category	Bid	Bname	Price
Costliest	103	Operating System Concepts	890.00
Cheapest	104	Advanced Database Management	450.00

```
2 rows in set (0.01 sec)
```



```
Query OK, 0 rows affected (0.01 sec)
```

6. Write a procedure to update the price of books by 25%, which are written by a particular author.

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE update_book_price_author(IN p_author_id INT)
-> BEGIN
->     UPDATE books b
->     JOIN writes w ON b.Bid = w.Bid
->     SET b.Price = b.Price * 1.25
->     WHERE w.Aid = p_author_id;
->
->     SELECT ROW_COUNT() AS updated_books_count;
-> END //
Query OK, 0 rows affected (0.00 sec)
```

```
mysql>
mysql> DELIMITER ;
mysql>
mysql> CALL update_book_price_author(308);
```

```
+-----+
| updated_books_count |
+-----+
|                    3 |
+-----+
1 row in set (0.01 sec)
```

Query OK, 0 rows affected (0.01 sec)

```
mysql> SELECT b.Bid, b.Bname, b.Price, w.Aid
-> FROM books b
-> JOIN writes w ON b.Bid = w.Bid
-> WHERE w.Aid = 308;
```

```
+-----+-----+-----+-----+
| Bid | Bname                                | Price  | Aid |
+-----+-----+-----+-----+
| 101 | Fundamentals of Database Systems    | 893.75 | 308 |
| 104 | Advanced Database Management        | 562.50 | 308 |
| 210 | Constitutional Law                  | 16875.00 | 308 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

7. Write a procedure to provide total money spent by a particular library in purchasing books in a particular month.

```
mysql> DELIMITER //
```

```
mysql>
```

```
mysql> CREATE PROCEDURE total_spent_by_month(  
->     IN p_lib_id INT,  
->     IN p_month_year VARCHAR(7), -- Pass as 'MM-YYYY', e.g. '08-2024'  
->     OUT p_total_spent DECIMAL(12, 2)  
-> )  
-> BEGIN  
->     SELECT COALESCE(SUM(totalcost), 0.00)  
->     INTO p_total_spent  
->     FROM purchase  
->     WHERE Lid = p_lib_id  
->         AND DATE_FORMAT(purchase_date, '%m-%Y') = p_month_year;  
-> END //
```

```
Query OK, 0 rows affected (0.00 sec)
```

```
mysql>
```

```
mysql> DELIMITER ;
```

```
mysql>
```

```
mysql> CALL total_spent_by_month(1, '08-2024', @total_spent);
```

```
Query OK, 1 row affected (0.00 sec)
```

```
mysql> SELECT @total_spent AS total_money_spent;
```

total_money_spent
3250.00

```
1 row in set (0.00 sec)
```

Conclusion:

I learned how to design, create, and execute parameterized stored procedures in MySQL using both IN and OUT parameters. I learned how encapsulating business logic directly inside the database simplifies complex multi-table joins, aggregations, and batch updates while enhancing execution efficiency. Furthermore, I learned the practical importance of maintaining referential integrity across relational schemas and handling environment-specific procedural delimiters properly.